

Online

Matemáticas Discretas

Ing. en Sistemas Computacionales

José Manuel Rodríguez Cantú

00612676

Tarea 4: Circuitos con compuertas lógicas y su aplicación

Módulo 4: Álgebra Booleana

Ana Laura Merino Díaz

Fecha:

2 de agosto de 2026

1. Introducción

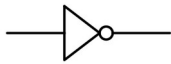
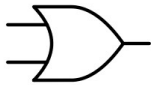
Las compuertas lógicas son los bloques básicos de los sistemas digitales. Son dispositivos que reciben una o más señales binarias y entregan una salida que depende de una operación del álgebra de Boole. Al combinarlas se pueden construir circuitos que resuelven problemas reales de control y automatización, desde una alarma de casa hasta el procesador de una computadora.

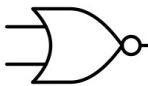
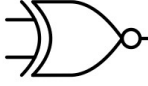
En el módulo anterior se elaboró la tabla de verdad de un problema de aplicación. En este reporte, que toma como base las lecturas del curso y algunas fuentes externas, se retoma ese mismo problema para llegar hasta el diseño del circuito y su comprobación por software: una puerta electrónica para una granja que solo debe abrirse con ciertas combinaciones de tres botones (A, B y C). Primero se describen las compuertas lógicas básicas, después se analiza el problema y se obtiene la función booleana a partir de la tabla de verdad, luego se simplifica la función y se diseña el circuito, y al final se comprueban los resultados con MATLAB.

2. Compuertas lógicas: características, interpretación y simbología

La Tabla 1 resume las compuertas lógicas básicas. Para cada una se indica su símbolo normalizado (norma ANSI/IEEE), su expresión booleana, la interpretación de su funcionamiento y un ejemplo sencillo de implementación en la vida cotidiana (Floyd, 2016; Tocci et al., 2017).

Tabla 1. Características, interpretación, simbología y ejemplos de las compuertas lógicas básicas.

Compuerta	Simbología	Expresión booleana	Características e interpretación	Ejemplo de implementación
NOT (inversor)		$S = A'$	Tiene una sola entrada. Invierte el valor de la entrada: si recibe 0 entrega 1, y si recibe 1 entrega 0.	Indicador de “falta de papel” de una impresora: la luz se enciende cuando el sensor deja de detectar hojas.
AND		$S = A \cdot B$	Producto lógico. La salida es 1 solo cuando todas sus entradas valen 1; basta un 0 para que la salida sea 0.	Prensa de mando bimanual: la máquina solo opera si el operador pulsa los dos botones al mismo tiempo.
OR		$S = A + B$	Suma lógica. La salida es 1 cuando al menos una de sus entradas vale 1; solo es 0 si todas las entradas son 0.	Timbre con botón en la puerta principal y en la trasera: suena al pulsar cualquiera de los dos.

Compuerta	Simbología	Expresión booleana	Características e interpretación	Ejemplo de implementación
NAND		$S = (A \cdot B)'$	AND negada. La salida es 0 únicamente cuando todas las entradas valen 1; en cualquier otro caso es 1.	Alarma de seguridad que se desactiva solo cuando los dos interruptores de protección están cerrados.
NOR		$S = (A + B)'$	OR negada. La salida es 1 únicamente cuando todas las entradas valen 0.	Indicador de “sistema en reposo”: se enciende solo cuando ningún motor está en marcha.
XOR		$S = A \oplus B$	OR exclusiva. La salida es 1 cuando sus entradas tienen valores diferentes, y 0 cuando son iguales.	Luz de escalera con dos apagadores: cambia de estado al accionar cualquiera de ellos.
XNOR		$S = (A \oplus B)' = A \odot B$	XOR negada; funciona como comparador de igualdad: la salida es 1 cuando las entradas son iguales.	Comparador digital que verifica si dos bits transmitidos coinciden (detección de errores).

3. Análisis del problema

En una granja se requiere una puerta electrónica que solo pueda abrirse cuando se pulse una determinada combinación de tres botones. Se definen las variables de entrada A, B y C, donde el valor 1 representa botón pulsado y el valor 0 botón en reposo. La salida S representa el estado de la puerta: $S = 1$ indica que la puerta se abre y $S = 0$ que permanece cerrada.

Las condiciones de apertura son dos: 1) que se pulse únicamente C, con A y B en reposo ($A = 0, B = 0, C = 1$); y 2) que se pulsen los tres botones a la vez ($A = 1, B = 1, C = 1$). En cualquier otra de las ocho combinaciones posibles la puerta debe permanecer cerrada. Esto da seguridad, porque es difícil que alguien que no conozca la clave atine a una de las dos combinaciones válidas.

Con base en el análisis combinatorio del módulo anterior, la Tabla 2 muestra la tabla de verdad completa del sistema con las $2^3 = 8$ combinaciones de entrada.

Tabla 2. Tabla de verdad de la puerta electrónica de la granja.

A	B	C	S	Interpretación
0	0	0	0	La puerta permanece cerrada
0	0	1	1	Condición 1: C pulsado, A y B en reposo
0	1	0	0	La puerta permanece cerrada
0	1	1	0	La puerta permanece cerrada

A	B	C	S	Interpretación
1	0	0	0	La puerta permanece cerrada
1	0	1	0	La puerta permanece cerrada
1	1	0	0	La puerta permanece cerrada
1	1	1	1	Condición 2: A, B y C pulsados

4. Función booleana y simplificación

La función booleana se obtiene de la tabla de verdad mediante la suma de productos (SOP): se toma un minitérmino por cada renglón donde $S = 1$. Los renglones activos son el m_1 ($A = 0, B = 0, C = 1$) y el m_7 ($A = 1, B = 1, C = 1$), por lo que la función canónica es:

$$S = A' \cdot B' \cdot C + A \cdot B \cdot C$$

Para simplificarla se aplica el álgebra de Boole. Ambos términos comparten la variable C , de modo que, por la propiedad distributiva, se factoriza:

$$S = C \cdot (A' \cdot B' + A \cdot B)$$

La expresión entre paréntesis es exactamente la definición de la compuerta XNOR (equivalencia): vale 1 cuando A y B son iguales (Mano & Ciletti, 2018). Por lo tanto, la función mínima queda:

$$S = C \cdot (A \odot B) = C \cdot (A \oplus B)'$$

El mapa de Karnaugh de la Tabla 3 confirma el resultado: los minitérminos m_1 y m_7 no son celdas adyacentes (difieren en dos variables), así que no pueden agruparse entre sí y la reducción se obtiene por factorización. El ahorro es claro: la forma canónica requiere dos inversores, dos compuertas AND de tres entradas y una OR (cinco compuertas), mientras que la forma simplificada solo necesita una XNOR y una AND (dos compuertas).

Tabla 3. Mapa de Karnaugh de la función $S(A, B, C)$.

A \ BC	00	01	11	10
0	0	1	0	0
1	0	0	1	0

5. Diseño del circuito lógico

La Figura 1 muestra el circuito que implementa directamente la función canónica: los inversores generan A' y B' , la compuerta AND 1 detecta la condición 1 ($A' \cdot B' \cdot C$), la AND 2 detecta la condición 2 ($A \cdot B \cdot C$) y la OR final activa la salida S cuando ocurre cualquiera de las dos.

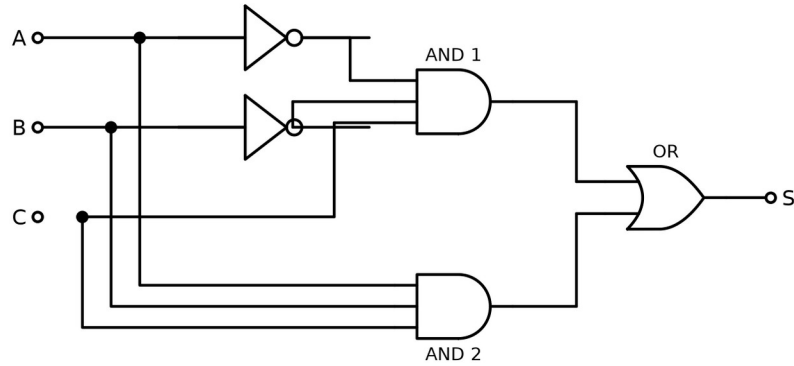


Figura 1. Circuito lógico de la función canónica $S = A'B'C + A \cdot B \cdot C$.

La Figura 2 presenta el circuito de la función simplificada: la compuerta XNOR compara los botones A y B y entrega 1 cuando coinciden; la compuerta AND combina ese resultado con C, de manera que la puerta solo se abre si, además de la coincidencia entre A y B, el botón C está pulsado. Este es el circuito propuesto como solución, por requerir menos componentes.

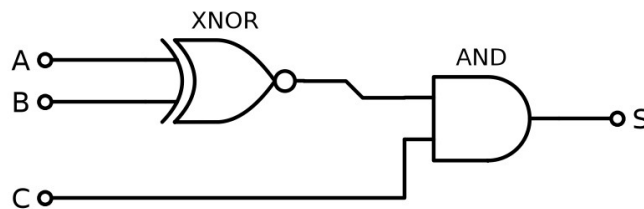


Figura 2. Circuito simplificado $S = C \cdot (A \odot B)$.

6. Comprobación de resultados en MATLAB

Para verificar el diseño se escribió un script en MATLAB que evalúa las ocho combinaciones de entrada tanto con la función canónica como con la simplificada, mediante los operadores lógicos del lenguaje (The MathWorks, 2024), imprime la tabla de verdad resultante y comprueba con `isequal` que ambas expresiones producen exactamente la misma salida.

```
% Comprobacion en MATLAB de la puerta electronica de la granja
% Apertura (S=1): 1) C pulsado, A y B en reposo  2) A, B y C pulsados
% Funcion canonica: S = A'·B'·C + A·B·C   Simplificada: S = C·(A XNOR B)
clc; clear;

% Las ocho combinaciones posibles de los botones A, B y C
A = [0 0 0 0 1 1 1 1]';
B = [0 0 1 1 0 0 1 1]';
C = [0 1 0 1 0 1 0 1]';

% Evaluacion de ambas expresiones booleanas
S_canonica = (~A & ~B & C) | (A & B & C);
S_simplificada = C & ~xor(A, B);

% Tabla de verdad completa
T = table(A, B, C, double(S_canonica), double(S_simplificada), ...
    'VariableNames', {'A', 'B', 'C', 'S_canonica', 'S_simplificada'});
disp(T)
```

```
% Verificacion de equivalencia entre ambas expresiones
if isequal(S_canonica, S_simplificada)
    disp('Ambas expresiones son equivalentes: el diseno del circuito es correcto.')
else
    disp('Las expresiones NO coinciden: es necesario revisar el diseno.')
end
```

Listado 1. Script comprobacion_puerta_granja.m.

Al ejecutar el script se obtiene en la ventana de comandos la siguiente salida:

A	B	C	S_canonica	S_simplificada
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	0	0
1	0	0	0	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Ambas expresiones son equivalentes: el diseno del circuito es correcto.

Listado 2. Salida del script en la ventana de comandos de MATLAB.

La tabla generada por MATLAB coincide renglón por renglón con la Tabla 2, y el mensaje final confirma que la función canónica y la simplificada son equivalentes.

7. Interpretación del resultado y conclusiones

El resultado tiene una interpretación física clara: la puerta de la granja se abre únicamente en 2 de las 8 combinaciones posibles. El botón C funciona como confirmación, ya que si no se pulsa la puerta no se abre. Además, los botones A y B deben estar en el mismo estado: los dos en reposo (condición 1) o los dos pulsados (condición 2). Si se pulsa solo A, solo B o combinaciones parciales como A y C o B y C, la puerta se queda cerrada.

La simplificación algebraica redujo el circuito de cinco compuertas a solo dos sin cambiar su comportamiento, y esto se comprobó con MATLAB. En la práctica, esto significa menor costo de componentes, menos conexiones que puedan fallar y una respuesta más rápida del circuito.

Se concluye que el álgebra de Boole y las compuertas lógicas permiten convertir paso a paso un requerimiento escrito, como las condiciones de apertura de una puerta, en un circuito digital que funciona y usa el menor número de componentes. Partir de la tabla de verdad garantiza que el diseño cumpla las especificaciones, la simplificación reduce el circuito y la comprobación con software valida el resultado antes de armar el circuito físico.

8. Enlace al video

[Insertar aquí el enlace al video con la explicación del desarrollo de la tarea.]

Referencias

Floyd, T. L. (2016). *Fundamentos de sistemas digitales* (11.^a ed.). Pearson Educación.

Mano, M. M., & Ciletti, M. D. (2018). *Diseño digital: con una introducción al Verilog HDL* (6.^a ed.). Pearson Educación.

The MathWorks. (2024). *Operadores lógicos: AND, OR, NOT* (documentación de MATLAB). <https://la.mathworks.com/help/matlab/logical-operations.html>

Tocci, R. J., Widmer, N. S., & Moss, G. L. (2017). *Sistemas digitales: principios y aplicaciones* (11.^a ed.). Pearson Educación.